CS 327 - Assignment 4

IR Generation

Members:

- Chaitanya Sharma 23110072
- Harshil Shah 23110132
- Jeet Joshi 23110148
- Akshat Saurin Shah

Introduction & Compilation Steps

Github repository: https://github.com/HarshilShah1804/FortranPlusPlus
File Structure:

```
.
├── ast
├── compile.sh
├── ir
│   ├── tac.c
│   └── tac.h
├── ir_outputs
│   ├── errors
├── lexer
│   ├── lexer.l
├── output.txt
├── parser
│   ├── parser.y
├── README.md
├── semantic
│   ├── sdt_semantic.c
│   ├── sdt_semantic.h
│   └── sdt_types.h
└── tests
    ├── errors
    │   ├── semantic
```

To compile the binary, run the script compile.sh which cleans any existing binary builds and recompiles it again. To run the compiler, use the command `./parser_out <file-path>`.

This would print the Three Address Code (TAC) Intermediate representation generated from the file and if the file gives some semantic error. It would return with zero-exit code and show the semantic errors. Otherwise, it prints the 3AC IR.

Part-1: Intermediate Code Generation

1. Design an Intermediate Representation (3AC)

To design an Intermediate Representation in the 3AC form, we have defined a custom struct named `TacInstr` which stores the `Op`, `arg1`, `arg2` and `result`. The definition of the struct can be found below:

```
typedef struct TacInstr {
   TacOp op;
   char *op_text;   /* operator token for TAC_BINARY/TAC_UNARY or raw text
*/
   char *arg1;
   char *arg2;
   char *result;
   struct TacInstr *next;
} TacInstr;
```

Here, the TacOp is the type of the operator which is used to handle printing of different operator types since all the different operators don't require both arguments to be printed.

2. Handle Simple Constructs

a. Arithmetic expressions

For generating IR for arithmetic expressions, we use syntax directed translation to create new instances of `TacInstr` structs. For example below is the yacc code along with SDT to generate IR for ADD_OP (`+`).

```
expression: expression ADD_OP expression
    {
    char *op = $2;
    char *temp = tac_new_temp();
    int ok = 1;
        if ($1 && $3) {
      tac_emit_binary(temp, $1->place, op, $3->place);
    }
```

```
    $$ = temp;
    free(op);
    }
```

So, what this SDT does is that it creates a new temporary variable for storing the result of addition using the operator, and the 2 operands (`$1`, `$3`). Then uses the utility `tac_emit_binary`, which is used to generate IR for binary operators (which require 2 operands), and store it as the result of `$$`. So, the result of the operation becomes the new temporary variable generated.

For unary operators, we have the `arg2` attribute as empty since they operate on a single argument.

b. Assignment Statements

For generating IR for assignment statements like `a = UNARY_OP / BINARY_OP / CONST`, because of bottom up parsing, we already have the value of RHS present with us already. So, we directly create a new IR instruction for the statement. Below is the yacc rule with SDT for emitting IR for assignment statements.

```
assignment_stmt: variable_ref ASSIGN expression
    {
    if ($1 && $3) {
      tac_emit_assign($1->place, $3->place);
    }
    $$ = tac_make_typed_value(NULL, SDT_TYPE_UNKNOWN);
    }
  ;
```

3. IR for a sample Program

Now, using the above 2 SDT rules for arithmetic operations and assignment statements, below is the generated IR for a small Fortran Code. The source code for the program is as follows:

```
x = 5. + 1.2
x = .5
x = 5e10 + 2e5
x = 3E-2 + 4E-4
x = 1.2e3
x = 5
```

Generated IR:

```
op                arg1                      arg2                result
---------------- -------------  ----------------  --
-----------------

+                 5.                        1.2                      t1
=                 t1                                                 x
=                 .5                                                 x
+                 5e10                      2e5                      t2
=                 t2                                                 x
+                 3E-2                      4E-4                     t3
=                 t3                                                 x
=                 1.2e3                                              x
=                 5                                                  x
```

From the IR, we can see that the temporary variables like `t1`, `t2`, etc. are getting generated for storing intermediate results of binary or unary operations as described above and then getting stored in the assignment statements. For example, if we analyze the IR, we see that the first instruction adds `5.` With `1.2` and store it in `t1` which gets assigned to `x` in the instruction 2. These 2 instructions represent the first statement in the source code which is `x = 5. + 1.2`.

Similarly, we transform each of the statement and expression in the source code by creating temporaries for binary/unary ops and using direct assignment in case of constants.

Part 2: Extended Intermediate Code Support

This sections demonstrates the implementation of decision making and iterative constructs like `if-else` statements and `for, do-while, while` loops.

1. If-Else statements

For generating IR for IF-ELSE, we use syntax directed translation to create labels and conditional jumps. Below is the yacc rule with SDT idea for IF (...) THEN ... ELSE ... ENDIF:

```
if_stmt: IF LPAREN expression RPAREN THEN
  {
    SdtIfCtx ctx = sdt_if_push();
    if ($3) {
      tac_emit_if_false_goto($3->place, ctx.else_label);
    }
  }
  executables if_else_opt ENDIF
  {
    SdtIfCtx ctx = sdt_if_pop();
```

```
    if (ctx.has_else) {
      tac_emit_label(ctx.end_label);
    } else {
      tac_emit_label(ctx.else_label);
    }
    $$ = NODE("if_stmt", 8, LEAF("IF"), LEAF("LPAREN"),
LEAF("expression"), LEAF("RPAREN"), LEAF("THEN"), $7, $8,
LEAF("ENDIF"));
  }
  ;
```

So, what this SDT does is that it creates labels for `else` and `end`, emit `ifFalseGoTo` which is used to jump to else condition in case the condition is false and place correct labels in the generated IR so the control flow is stitched correctly.

2. Do Loops

For generating IR for counted DO loops, SDT creates loop start/end labels and emits loop control operations. Below is the `yacc` rule for handling `do loops`:

```
do_stmt: DO IDENTIFIER ASSIGN expression COMMA expression step_opt
  {

    SdtForCtx ctx = sdt_for_push($2, $7 ? $7->place : NULL);
    if ($4) {
      tac_emit_assign($2, $4->place);
    }
    tac_emit_label(ctx.start_label);
    {
      char *cmp = tac_new_temp();
      tac_emit_binary(cmp, $2, "<=", $6 ? $6->place : "0");
      tac_emit_if_false_goto(cmp, ctx.end_label);
    }
  }
  executables END DO
    {
    SdtForCtx ctx = sdt_for_pop();
    const char *step = (ctx.step_value && ctx.step_value[0] != '\0')
? ctx.step_value : "1";
    char *next = tac_new_temp();
    tac_emit_binary(next, ctx.var_name, "+", step);
    tac_emit_assign(ctx.var_name, next);
    tac_emit_goto(ctx.start_label);
    tac_emit_label(ctx.end_label);
```

```
    $$ = NODE("do_stmt", 10, LEAF("DO"), LEAF("IDENTIFIER"),
LEAF("ASSIGN"), LEAF("expression"), LEAF("COMMA"),
LEAF("expression"), LEAF("step_opt"), $9, LEAF("END"), LEAF("DO"));
    }
  ;
```

So, this SDT initializes loop variable, checks loop continuation condition, emits the executable body, Increments loop variable by step (1 if omitted), and jumps back to start until condition fails.

Similarly, we also handle the `do-while` loops in a similar manner.

Sample Program Translation

Source Program:

```
program if_while_sdt
 integer :: i, sum
 i = 0
 sum = 0
 do while (i < 5)
   if (i == 2) then
     sum = sum + 10
   else
     sum = sum + i
   end if
   i = i + 1
 end do
end program if_while_sdt
```

Generated IR:

```
TAC Output:
op              arg1                    arg2                    result
--------------- ----------------------- ----------------------- -----------------------
=               0                                               i
=               0                                               sum
label                                                           L1
<               i                       5                       t1
ifFalseGoto     t1                      L2
==              i                       2                       t2
ifFalseGoto     t2                      L3
+               sum                     10                      t3
=               t3                                              sum
goto            L4
label                                                           L3
+               sum                     i                       t4
=               t4                                              sum
label                                                           L4
+               i                       1                       t5
=               t5                                              i
goto            L1
label                                                           L2
```

The generated IR starts by assigning `i` and `sum` variables to `0`. Then it creates a label `L1`, which marks the start of the do loop. Then, it calculates the condition `(i<5)` and stores it in the temporary variable `t1`. Then, if `t1` is false i.e. `i >= 5` implying loop execution is completed, it goes to the label `L2` which marks the end of the do loop. Otherwise it executes the statements in the body of the loop.

In the loop body, it creates another logical variable to compare `i == 2` and store it in `t2`, which represents the condition for the `if` block `if(i==2)` and again emit a `ifFalseGoTo` instruction to mark the skipping of `if` statement. Otherwise, it executes the `if` body by adding `10` to `sum` variable.

Similarly, we represent the other `if` loop also in the IR.

Part 3: Code Output Format

We have described almost the complete flow of the output format of generating the temporary variables and emitting the labels and goto's in the above 2 sections. In this section, we describe the utilities used to print the generated IR in the tabular format as specified in the class slides.

Below is the utility `tac_print` that we use to print the generated 3AC IR in the tabular format.

```
void tac_print(FILE *out)
{
   if (!out) {
       out = stdout;
```

```
    }

    fprintf(out, "%-16s %-24s %-24s %-24s\n", "op", "arg1", "arg2",
"result");
    fprintf(out, "%-16s %-24s %-24s %-24s\n", "----------------",
"------------------------", "------------------------",
"------------------------");

    for (TacInstr *cur = g_head; cur; cur = cur->next) {
        const char *op = "";
        const char *arg1 = "";
        const char *arg2 = "";
        const char *result = "";

        switch (cur->op) {
            case TAC_ASSIGN:
                op = "=";
                arg1 = cur->arg1 ? cur->arg1 : "";
                result = cur->result ? cur->result : "";
                break;
            case TAC_BINARY:
                op = cur->op_text ? cur->op_text : "binary";
                arg1 = cur->arg1 ? cur->arg1 : "";
                arg2 = cur->arg2 ? cur->arg2 : "";
                result = cur->result ? cur->result : "";
                break;
            case TAC_UNARY:
                op = cur->op_text ? cur->op_text : "unary";
                arg1 = cur->arg1 ? cur->arg1 : "";
                result = cur->result ? cur->result : "";
                break;
            case TAC_IF_FALSE_GOTO:
                op = "ifFalseGoto";
                arg1 = cur->arg1 ? cur->arg1 : "";
                arg2 = cur->result ? cur->result : "";
                break;
            case TAC_GOTO:
                op = "goto";
                arg1 = cur->result ? cur->result : "";
                break;
```

```
            case TAC_LABEL:
                op = "label";
                result = cur->result ? cur->result : "";
                break;
            case TAC_RAW:
            default:
                op = "raw";
                result = cur->op_text ? cur->op_text : "";
                break;
        }

        fprintf(out, "%-16s %-24s %-24s %-24s\n", op, arg1, arg2, result);
    }
}
```

The utility uses the `Op` field in the TAC struct which represents the type of the instruction to decide the number of arguments which would be present in the instruction based on the type of the instruction and then print it. For example, for `TAC_UNARY` type instructions, we only have the `arg1` field and the `arg2` is empty since it works on a single operand.
Note: We use the `raw` instruction to represent the calls like `print` or `function calls` which are out of scope for this assignment and can be handled as a future work to make the compile more robust and support more constructs.

We were asked to generate IRs for at least 10 simple programs in the assignment, 2 of which are already shown in the above sections. The other generated IRs along with their original source code can be found in the ir_outputs directory of the Github repository. Also, to generate the IR for any other program, use the command `./parser_out <file-path>`. It would generate the IR for the program, if the program is syntactically correct otherwise, it would generate parse errors.

Part 4: Error Handling

The parse errors for invalid syntax of the code were already submitted as a part of Assignment 3 - Syntax Analysis. This assignment introduces error handling for invalid constructs and semantics of the program. Below is the list of invalid code constructs that we currently handle using the error handling:

1. **Duplicate Declaration of Variables**
   Occurs when the same variable is declared more than once in the same scope. This leads to ambiguity about which declaration should be used.
2. **Type Mismatch in Declaration**
   Happens when a variable is declared with a type that conflicts with its initialization value

or intended usage. For example, assigning a string value to an integer variable during declaration.

3. **Use of Undeclared Variable**
   Triggered when a variable is used before it has been declared. The compiler cannot determine its type or memory location.
4. **Type Mismatch in Assignment**
   Occurs when the value assigned to a variable does not match its declared type. For instance, assigning a floating-point value to an integer variable without conversion.
5. **Condition in `if` or `do-while` Not Logical**
   Happens when the condition expression does not evaluate to a boolean/logical type. Control flow statements require conditions that result in true or false.
6. **Undeclared `do`-loop Variable**
   Occurs when the loop control variable in a `do` loop is used without prior declaration. This makes it impossible to track or update the loop variable correctly.
7. **Non-integer Values in `do`-loop Bounds**
   Triggered when the start, stop, or step values in a `do` loop are not integers. Loop iteration counts must be integer-based for proper execution.
8. **Step Value of `do`-loop is Zero**
   Occurs when the step/increment value is zero, which would cause an infinite loop. Such cases are flagged as invalid.
9. **Type Mismatch in Comparison**
   Happens when incompatible types are compared, such as comparing a string with a number. This can lead to undefined or incorrect results.
10. **Invalid Operands for Arithmetic and Logical Operators**
    Occurs when operators are applied to unsupported operand types, like adding a boolean to an integer or applying logical operators to non-boolean values.
11. **Accessing Elements of a Scalar Variable**
    Triggered when a non-array (scalar) variable is used as if it were an array (e.g., using indexing). Scalars do not support element access operations.

For creating this error handling mechanism, we create a symbol table where we store each of the variables along with a few of its attributes. Below is the struct type which we use to store the variables:

```
typedef struct SdtSymbol {
   char *name;
   SdtType type;
   int is_array;
   struct SdtSymbol *next;
} SdtSymbol;
```

So, each symbol stores its name, variable type and a boolean expression indicating whether it is an array variable or not. We store them in the form of a linked list. But it can be optimized to use some kind of a map which makes accessing the symbols faster.

So, whenever we create a variable either by declaration or a temporary, we create an entry for it in the Symbol Table which helps in handling invalid constructs to stop invalid expression with type mismatch, scalar indexing etc.

The complete implementation details of the semantic handling can be found in semantic/README.md file in the github repository. The generated error outputs can be found in the ir_outputs/errors directory along with the source code and the error message for the failure.

Miscellaneous

The rest of the report including the end-to-end pipeline documentation and work distribution for the implementation can be found in the README.md file at the repo.